

Planificación automática aplicada al Senku mediante Beam Search guiado por la función pagoda

Pablo Reche Gabaldón

Dpto. Ciencias de la Computación e Inteligencia Artificial
Universidad de Sevilla
Sevilla, España
pabrecgab@alum.us.es

Sol Villegas Charlo

Dpto. Ciencias de la Computación e Inteligencia Artificial
Universidad de Sevilla
Sevilla, España
solvilcha@alum.us.es

Resumen—En este trabajo planteamos el juego del Senku (o solitario) como un problema de planificación automática clásica. El dominio se modela en PDDL con el estilo de la Práctica 4, y sobre esa base construimos un sistema en Python que recibe dos ficheros .pddl arbitrarios (dominio e instancia) y los resuelve para las cinco variantes del enunciado. La parte común se cubre con búsqueda en anchura; para la ampliación de junio implementamos *beam search* guiado por la *función pagoda*, con dos extensiones (reinicios estocásticos y anchura iterativa) y un total de cinco heurísticas comparadas. Como línea de base usamos Fast Downward a través de la biblioteca *unified-planning*.

El resultado más llamativo es que la pagoda, pese a ser admisible, no contiene suficiente información para guiar el haz: ni siquiera con $\beta = 2000$, veinte reinicios y casi 10^6 nodos explorados se encuentra un plan. Como evidencia adicional, ninguna de las 80797 partidas aleatorias que lanzamos sobre la cruz inglesa terminó con una sola pieza. La clave fue cambiar de heurística: una basada en *conectividad* (minimizar el número de componentes conexas de piezas) combinada con *beam search iterativo* resuelve la cruz inglesa en 31 movimientos y, en general, las tres variantes obligatorias. Un barrido exhaustivo de la posición del hueco inicial caracteriza empíricamente la solubilidad de cada tablero. Por su parte, Fast Downward resuelve V3 y V4 rápidamente pero falla en V1, V2 y V5 incluso con timeouts de 300 s, lo que sitúa a nuestro beam search en un rol complementario al planificador genérico, no de competidor.

Palabras clave—Inteligencia Artificial, planificación automática, PDDL, beam search, función pagoda, Senku.

I. INTRODUCCIÓN

El *Senku* —o *solitario*, *peg solitaire*, *Hi-Q*— es un juego de un solo jugador [1]. El tablero es una rejilla con M casillas y $M-1$ piezas; queda un hueco. En cada turno se coge una pieza, se salta sobre otra adyacente (en vertical u horizontal) y se aterriza en el hueco contiguo, retirando la pieza saltada. La meta clásica es dejar una pieza en la posición del hueco original; hay también versiones relajadas en las que basta con quedarse con una pieza en cualquier punto. La variante más estudiada es la *cruz inglesa* de 33 casillas, sobre la que existe una literatura extensa desde los años setenta que culmina en la monografía de Beasley [5].

Aunque las reglas son sencillas, el problema computacional no lo es. La cruz inglesa tiene del orden de $1,87 \cdot 10^8$ posiciones alcanzables, y de éstas sólo $5,68 \cdot 10^5$ llevan a la solución canónica (una pieza en el centro). Es una densidad próxima a $3 \cdot 10^{-3}$, y a eso se suma que abundan los estados “trampa”

(piezas aisladas que ya no se pueden saltar). Para una búsqueda ciega es poco; para una heurística mediocre, también: hace falta una señal con bastante capacidad de discriminación.

Como las acciones son discretas, deterministas y observables, el Senku encaja en planificación clásica de manera bastante natural [2]: cada estado es un nodo y cada salto válido una arista. Trabajar en PDDL tiene además la ventaja de separar el modelo de la técnica de resolución: con el mismo fichero podemos comparar nuestro beam search en Python frente a un planificador genérico como Fast Downward.

A. Objetivos y contribuciones

El objetivo del trabajo es montar un sistema completo de resolución del Senku basado en planificación automática, que cubra a la vez la parte común y la ampliación de junio. Lo que hemos hecho se puede resumir así:

- Un dominio PDDL del Senku en nivel STRIPS, escrito de forma genérica para que sirva para los cinco tableros sin tocar el dominio.
- Un sistema en Python construido sobre *unified-planning* (al estilo de la Práctica 4) que lee dos ficheros PDDL cualesquiera (dominio e instancia) y los resuelve con tres motores: BFS clásica, *beam search* con la heurística que se le indique, o Fast Downward como línea de base.
- Tres versiones de *beam search*: la básica del enunciado, una con reinicios estocásticos para diversificar el desempate, y una iterativa que prueba anchuras crecientes.
- Cinco heurísticas comparadas: dos asignaciones de pagoda (uniforme y clásica) y tres complementarias (aislamiento, compacidad y conectividad).
- Un estudio experimental sobre las cinco variantes: comparativa contra Fast Downward, diagnóstico de por qué la pagoda no basta como guía de la búsqueda, y un barrido sistemático de la posición del hueco inicial bajo simetría del tablero. De ese barrido sale una caracterización empírica de qué tableros son resolubles desde qué huecos.

El documento sigue una estructura clásica. La Sección II revisa los preliminares (PDDL, *unified-planning*, beam search y función pagoda). La Sección III explica el dominio PDDL, la representación interna, las heurísticas y los algoritmos, justificando las decisiones de diseño. La Sección IV

reúne los resultados de los experimentos, y la Sección V cierra con las conclusiones y líneas futuras.

II. PRELIMINARES

A. PDDL y unified-planning

PDDL (*Planning Domain Definition Language*) es el estándar de facto para describir problemas de planificación automática clásica [2]. Su especificación divide cada problema en dos partes: el *dominio*, que recoge los predicados y los esquemas de acción independientes de la instancia concreta, y el *problema*, que fija los objetos, el estado inicial y la meta. La versión 1.2 establece la sintaxis mínima —el subconjunto STRIPS— y es la que se utiliza en este trabajo, dado que el Senku no requiere ni cantidades numéricas ni efectos condicionales. Versiones posteriores (2.1, 2.2 y 3.0) añaden tipos, flujos numéricos, axiomas y restricciones temporales, soportadas también por la biblioteca usada.

La biblioteca unified-planning [3] es una capa de abstracción Python sobre planificadores externos. Nos sirve para tres cosas: leer y escribir ficheros PDDL (PDDLReader, PDDLWriter), construir problemas desde código con Problem, Fluent, UserType y InstantaneousAction, y lanzar planificadores externos vía OneshotPlanner. La gracia es que el mismo problema se le puede pasar a varios planificadores sin reescribir nada. En el trabajo la usamos siguiendo la misma metodología que en la Práctica 4, y la conectamos con Fast Downward [4] a través de up-fast-downward.

B. Fast Downward

Fast Downward [4] es uno de los planificadores independientes del dominio más usados en planificación clásica. Su arquitectura tiene tres bloques: traduce el dominio PDDL a una representación interna llamada SAS⁺, donde cada variable agrupa hechos mutuamente excluyentes; aplica un preprocesado que detecta invariantes y simplifica el modelo; y, sobre eso, ejecuta un motor de búsqueda configurable que combina algoritmos (A*, greedy, satisficing) con heurísticas avanzadas (FF, h^{max}, h^{add}, h^{LM-cut}, etc.).

Nosotros lo usamos como línea de base contra la que comparar nuestro beam search. Tiene la ventaja de ser completo en problemas finitos: si responde SOLVED_SATISFICING hay plan, y si responde UNSOLVABLE_INCOMPLETELY no lo hay. Eso nos da un diagnóstico fiable cuando queremos saber si una variante del Senku es estructuralmente irresoluble o si simplemente nuestro beam no la ha sabido resolver.

C. Beam search

Beam search es una variante de búsqueda en anchura con memoria limitada. Se usa mucho en problemas con espacios de estados grandes (traducción automática neuronal, reconocimiento del habla, etc.). La idea es simple: en cada nivel se mantienen sólo los β estados con mejor heurística y se tira el resto sin posibilidad de recuperarlos. El parámetro β (la *anchura del haz*) regula el trade-off entre memoria/tiempo y cobertura.

A diferencia de BFS, beam search no es óptimo y tampoco es completo: la única ruta a la meta puede haberse descartado en algún nivel intermedio. Aun así resulta atractivo porque su consumo de memoria es lineal en β y, con una heurística que dé información útil, encuentra planes muy rápido.

La literatura recoge varias modificaciones: *beam stochastic search* (selección probabilística en vez de top- β), *beam stack search* (con backtracking) o el *iterative widening beam search* (anchura creciente entre iteraciones). En el trabajo implementamos la versión clásica del enunciado, una con reinicios estocásticos para diversificar el desempate y una iterativa que va subiendo β .

D. Función pagoda

Las *funciones pagoda* son una técnica clásica introducida por Conway [5] para analizar el peg solitaire. Su formalización es la siguiente: una asignación de pesos $\pi : \text{Casillas} \rightarrow \mathbb{Z}_{\geq 1}$ es una *función pagoda* si para toda terna (a, b, c) de casillas consecutivas y alineadas se cumple $\pi(a) + \pi(b) \geq \pi(c)$. Bajo esa condición, el valor pagoda de un estado $\Pi(s) = \sum_{p \in s} \pi(p)$ es monótono no creciente con cada movimiento legal: si la terna (a, b, c) define un salto, la diferencia entre el estado resultante y el anterior es

$$\Pi(s') - \Pi(s) = \pi(c) - \pi(a) - \pi(b) \leq 0. \quad (1)$$

De aquí sale directamente una heurística admisible: si el valor pagoda del estado actual es menor que el valor pagoda objetivo, la meta es inalcanzable; si no, el excedente $\Pi(s) - \Pi^*$ es el mínimo número de unidades de pagoda que aún hay que “gastar” para llegar [5].

La función pagoda no es única. De hecho hay infinitas asignaciones válidas para un mismo tablero, y de la que se elija depende cuánto discrimine la heurística. En este trabajo probamos dos: una uniforme ($\pi \equiv 1$), que hace que el valor pagoda coincida con el número total de piezas, y la clásica, definida a partir de la distancia de Chebyshev al baricentro y que da más peso a las casillas centrales.

III. METODOLOGÍA

A. Dominio PDDL del Senku

El dominio (Fig. 1) tiene tres predicados: *ocupada*, *vacía* y *salto*. Los dos primeros representan el estado dinámico de cada casilla; el tercero codifica la *geometría* del tablero. La idea es que *salto* no aparece en las acciones como algo que cambie, sino como hechos estáticos que se declaran en el `:init` de cada problema. Eso es lo que permite tener un solo dominio común para todas las topologías: para cambiar de tablero basta con cambiar los hechos *salto* del problema, sin tocar el dominio.

```

(define
  (domain senku)
  (:requirements :strips)
  (:predicates
    (ocupada ?p)
    (vacía ?p)
    (salto ?desde ?sobre ?hasta))
  (:action mover
    :parameters (?desde ?sobre ?hasta)
    :precondition (and
      (salto ?desde ?sobre ?hasta)
      (ocupada ?desde) (ocupada ?sobre)
      (vacía ?hasta))
    :effect (and
      (not (ocupada ?desde)) (vacía ?desde)
      (not (ocupada ?sobre)) (vacía ?sobre)
      (not (vacía ?hasta)) (ocupada ?hasta))))

```

Fig. 1. Dominio PDDL común a las cinco variantes del Senku.

La única acción del dominio es mover: exige pieza en origen y en la casilla intermedia, hueco en el destino y un hecho salto que valide la geometría. El efecto invierte el estado de las tres casillas implicadas.

Cada problema se genera con `python -m senku.src.cli generar`, que escribe los hechos `ocupada`, `vacía` y `salto` correspondientes al estado inicial y la conjunción que define la meta. El número de hechos `salto` es del orden de cuatro veces el número de casillas (un sentido por cada dirección legal); para la cruz inglesa, por ejemplo, salen 76 hechos `salto`.

1) *Alternativas que descartamos*: Antes de quedarnos con esta formulación barajamos otras tres:

- 1) Codificar la geometría dentro del dominio con tipos por fila y columna, usando predicados como `misma_fila` y `misma_columna`. Funciona, pero hace que el dominio dependa de la geometría del tablero y se pierde la generalidad.
- 2) Tener tres acciones distintas (`mover-horizontal`, `mover-vertical-arriba`, `mover-vertical-abajo`) con predicados `adyacente-h` y `adyacente-v`. Eso triplica el número de acciones sin ganar nada en expresividad.
- 3) Modelarlo con flujos numéricos y una métrica de optimización (PDDL 2.1). Para la versión satisficing del problema es exagerado y complica innecesariamente la especificación.

La opción que escogimos —`salto` como hecho estático del `:init`— nos parecía la que mejor equilibra simplicidad, generalidad y compatibilidad con planificadores STRIPS.

B. Variantes consideradas

El enunciado define cinco variantes del Senku en su Figura 3 (Sección 3.2) y obliga a resolver las variantes 1, 3 y 5. La Tabla I resume las dimensiones de cada una.

TABLA I
VARIANTES DEL SENKU IMPLEMENTADAS SEGÚN LA FIGURA 3 DEL ENUNCIADO.

#	Nombre	Caja	Casillas	Obligatoria
1	Octógono	7×7	37	sí
2	Cruz griega grande	9×9	45	no
3	Cruz asimétrica	8×8	39	sí
4	Cruz griega clásica (inglesa)	7×7	33	no
5	Rombo / diamante	9×9	41	sí

Cada variante se especifica por la distribución de casillas por fila y la posición del hueco inicial. Estas son las definiciones concretas, con coordenadas 0-indexed:

- **Variante 1 (octógono, 37 casillas)**: caja 7×7, casillas por fila {3, 5, 7, 7, 7, 5, 3}, hueco inicial en (2,3) —una fila por encima del centro geométrico (3,3), que sí contiene ficha—. Es un octógono inscrito en una rejilla 7×7 con triángulos en las cuatro esquinas recortados.
- **Variante 2 (cruz griega grande, 45 casillas)**: caja 9×9, casillas por fila {3, 3, 3, 9, 9, 9, 3, 3, 3}. Cruz simétrica con brazos de 3 casillas de ancho y 9 de largo (3 antes y 3 después del cuadrado central). Hueco en el centro (4,4).
- **Variante 3 (cruz asimétrica, 39 casillas)**: caja 8×8, casillas por fila {3, 3, 3, 8, 8, 8, 3, 3}. Es una cruz *no simétrica*: el brazo vertical (columnas 2–4) tiene 2 casillas a la izquierda y 3 a la derecha del eje, y el brazo horizontal (filas 3–5) tiene 3 filas por encima y 2 por debajo. Hueco en (4,3).
- **Variante 4 (cruz griega clásica, 33 casillas)**: caja 7×7, casillas por fila {3, 3, 7, 7, 7, 3, 3}. Es el Senku “inglés” estándar, ampliamente estudiado en la literatura clásica del *peg solitaire* [5]. Hueco en el centro (3,3).
- **Variante 5 (rombo, 41 casillas)**: caja 9×9, casillas por fila {1, 3, 5, 7, 9, 7, 5, 3, 1}. Diamante perfecto definido por $|r - 4| + |c - 4| \leq 4$. Hueco en el centro (4,4).

Las variantes 2 y 4 no son obligatorias, pero las hemos incluido para tener material adicional con el que experimentar. La V4 (cruz inglesa) es de hecho el Senku clásico, así que sirve como referencia histórica; y la V2 (cruz griega grande) viene bien como ejemplo de tablero simétrico de mayor tamaño.

C. Representación interna

Cada estado se representa con un `frozenset` que contiene las coordenadas (*fila*, *columna*) de las casillas ocupadas. Es lo mínimo que hace falta: las casillas vacías se calculan como diferencia entre el conjunto total y el estado actual. Optamos por `frozenset` frente a `tuple`, `bitmask` o matriz binaria por tres motivos:

- Es *hashable*, así que podemos usar los estados como claves de diccionario para el conjunto de visitados y para reconstruir el camino.
- La igualdad entre estados es $O(1)$ amortizado una vez calculado el hash, lo que importa porque el haz puede contener miles de estados a la vez.

- Las transiciones son baratas: pasar de s a s' es $s' = (s \setminus \{a, b\}) \cup \{c\}$, en $O(1)$ amortizado.

Una alternativa razonable habría sido usar un entero como *bitmask* de longitud igual al número de casillas. En CPython sería algo más rápido por usar operaciones nativas sobre enteros largos, pero pierde legibilidad y complica la reutilización del código para tableros arbitrarios.

Sobre esta representación, el módulo `ProblemaSenku` guarda también la lista de saltos posibles y la meta. Su método `sucesores` genera todos los pares (movimiento, s') aplicables a un estado en $O(|\text{saltos}|)$, donde $|\text{saltos}|$ es el número de ternas `salto` del tablero.

D. Heurísticas implementadas

Sobre la representación anterior hemos implementado cinco heurísticas que cubren el espectro entre admisibilidad estricta (cota inferior formal, peor guía) e informatividad empírica (mejor guía pero sin garantías):

1) *Pagoda uniforme*: $\pi \equiv 1$. El valor pagoda coincide con el número de piezas vivas, y la heurística asociada $h(s) = \max(0, |s| - |S^*|)$ es el número de piezas excedentes respecto a la meta. Es admisible (cada movimiento elimina exactamente una pieza, así que se requieren al menos $|s| - |S^*|$ jugadas) y trivialmente válida en cualquier tablero.

2) *Pagoda clásica*: $\pi(r, c) = \max(1, 8 - d_\infty(r, c))$, donde d_∞ es la distancia de Chebyshev al baricentro del tablero, normalizada para que el centro reciba el peso 8 y las casillas periféricas el peso 1. Premia las piezas centrales y satisface la cota $a + b \geq c$ en todas las ternas alineadas (verificado por test automático sobre las cinco variantes). Es admisible y más discriminativa que la uniforme porque distingue piezas según su posición.

3) *Heurística de aislamiento*: Cuenta el número de piezas *aisladas* en el estado, definidas como piezas que no pueden participar en ningún movimiento posible (ni como origen, ni como pieza saltada, ni como destino). Una pieza aislada es un predictor fuerte de callejón sin salida: nunca podrá eliminarse, por lo que el estado no puede llegar a 1 pieza salvo que ya quede solo esa.

4) *Heurística compuesta*: Combina las tres anteriores de forma lineal, con pesos $w_{\text{pag}} = 1$, $w_{\text{aisl}} = 1000$ y $w_{\text{comp}} = 0,1$. La *compacidad* es la suma de distancias de Chebyshev de las piezas al centroide del objetivo. Deja de ser admisible (los dos últimos términos pueden sobreestimar la distancia a la meta), pero ayuda a beam search a evitar callejones sin salida. El peso tan alto del término de aislamiento hace que la heurística esté dominada de hecho por la detección de estados “muertos”.

5) *Heurística de conectividad*: Es la heurística que acabó marcando la diferencia. Ordena los estados por el número de *componentes conexas* de piezas (con adyacencia ortogonal), penalizando además las piezas aisladas y, en menor medida, el número total de piezas. La intuición: para terminar con una sola pieza, el conjunto tiene que llegar cohesionado al final, en una única componente; dos grupos separados por huecos imposibles de cruzar nunca podrán fusionarse. En

modo estricto añadimos también un término de atracción hacia la casilla objetivo cuando ya queda poco. Formalmente:

$$h_c(s) = 50 \cdot \kappa(s) + 20 \cdot \alpha(s) + |s|, \quad (2)$$

donde $\kappa(s)$ es el número de componentes conexas de s por adyacencia ortogonal y $\alpha(s)$ es el número de piezas aisladas. Los coeficientes están elegidos para que κ mande, α desempate y $|s|$ actúe de tercer criterio. No es admisible, pero es la única con la que beam search resuelve la cruz inglesa (Sección IV).

E. Algoritmos de búsqueda

Sobre el motor anterior implementamos en Python la BFS clásica (con conjunto global de visitados y cota opcional de nodos) para la parte común del trabajo, y el *beam search* de la convocatoria de junio. El pseudocódigo de este último está en la Fig. 2.

beam-search(P, h, β)

Entrada: problema P , heurística h , anchura β

Salida: plan, o fallo

```

1   $F \leftarrow \{s_0\}, V \leftarrow \{s_0\}$ 
2  mientras  $F \neq \text{fallo}$ 
3       $C \leftarrow []$ 
4      para cada  $s \in F$  hacer
5          para cada  $(a, s') \in \text{SUCESORES}(s)$  hacer
6              si  $P.\text{meta}(s')$  entonces devolver plan hasta } s'
7              si  $s' \notin V$  entonces
8                   $V \leftarrow V \cup \{s'\}$ 
9                   $C.\text{añade}(\langle h(s'), s' \rangle)$ 
10             ordenar  $C$  por } h ascendente
11              $F \leftarrow \text{primeros } \beta \text{ elementos de } C$ 
12 devolver fallo
```

Fig. 2. *Beam search* con conjunto de visitados.

Además del beam clásico hemos implementado dos variantes.

La primera, *beam search con reinicios estocásticos*, lanza el algoritmo k veces con semillas distintas para desempatar entre estados con la misma heurística, y se queda con el primer éxito. Es una forma estándar de mitigar la incompletud del beam search: si la primera ejecución se atasca en un callejón sin salida concreto, las siguientes exploran otras ramas con alta probabilidad. El desempate se hace sumándole a la clave de ordenación un valor pseudoaleatorio entre 0 y 1.

La segunda, *beam search iterativo*, va probando anchuras crecientes $\{\beta_1 < \beta_2 < \dots\}$ (por defecto $\{100, 300, 800, 1500\}$). Si con β_i y los k reinicios no aparece plan, se pasa a β_{i+1} . Así se evita pagar el coste de un haz grande cuando la instancia es fácil con uno pequeño, sin perder cobertura para las instancias más difíciles. La idea es análoga al *iterative deepening* de DFS, pero aplicada a la anchura del haz en vez de a la profundidad.

El Resultado que devuelven estas búsquedas tiene además un atributo `min_piezas_alcanzadas`: el mínimo número de piezas que se ha visto en algún estado de la

búsqueda. Es un diagnóstico útil cuando no se llega a la meta: por ejemplo, que se haya quedado en 2 piezas sugiere que estuvimos a un paso, mientras que quedarse en 4 o más indica que la instancia es estructuralmente más difícil.

F. Sistema de entrada PDDL

El requisito de la convocatoria —“*recibir dos archivos .pddl, uno con el dominio y otro con una instancia del problema*”— lo cumplimos con el módulo `lector_pddl.py`. El camino principal usa `unified-planning`, igual que en la Práctica 4:

```
lector = PDDLReader()
problema_up = lector.parse_problem(
    str(ruta_dominio), str(ruta_problema))
```

A partir del objeto que devuelve la biblioteca recorreremos los valores iniciales y la meta para construir la representación interna `ProblemaSenku` que esperan los algoritmos de búsqueda. Como red de seguridad por si `unified-planning` no está disponible, incluimos un parser propio basado en S-expresiones que sólo entiende el dominio `Senku`.

IV. RESULTADOS

A. Línea de base: Fast Downward

Antes de probar nada nuestro, lanzamos Fast Downward sobre cada variante a través de `unified-planning` con la misma receta de la Práctica 4 (Tabla II). Fast Downward resuelve las dos variantes con geometría de cruz desde sus huecos nominales: la V3 (cruz asimétrica) en 37 movimientos y 8,1 s, y la V4 (cruz inglesa) en 31 movimientos y 17,9 s. En cambio, V1 (octógono), V2 (cruz griega grande) y V5 (rombo) no se resuelven aunque le demos margen: probamos hasta 300 s de timeout y también huecos iniciales alternativos para V1, sin éxito.

Atribuimos esa dificultad a la geometría. El octógono, la cruz griega grande y el rombo tienen formas más densas que las cruces simétricas; eso aumenta el factor de ramificación y dispersa la información que utiliza la heurística por defecto de FD (FF, basada en relajación delete). Es un comportamiento que aparece descrito en la literatura sobre *peg solitaire* y problemas combinatorios parecidos: los planificadores satisficing genéricos se quedan cortos cuando la estructura del estado es muy densa.

TABLA II
RESOLUCIÓN CON FAST DOWNWARD.

Var.	Estado	Movs.	T (s)
1	TIMEOUT	–	>300
2	TIMEOUT	–	>60
3	SOLVED_SATISFICING	37	8,1
4	SOLVED_SATISFICING	31	17,9
5	TIMEOUT	–	>60

Datos tras aplicar el parche `parche_fd.py`, que neutraliza un bug de `up-fast-downward 0.5.2` al decodificar la salida de FD en Windows.

B. BFS y beam search propios

La Tabla III resume cómo se comportan nuestras implementaciones sobre los mismos ficheros PDDL, cargados con `unified-planning`. La BFS la limitamos a 30 000 nodos para que el experimento no se alargue. El beam search lo lanzamos con $\beta = 200$, tres reinicios estocásticos y un máximo de 80 iteraciones por intento, bajo dos heurísticas: pagoda clásica y compuesta.

TABLA III
BFS Y BEAM SEARCH PROPIOS (SIN ALCANZAR LA META EN NINGUNA VARIANTE).

Var.	Alg.	Heur.	Nodos	T (s)
1	BFS	–	30 001	2,76
1	Beam	pagoda	12 576	2,04
1	Beam	compuesta	14 924	9,55
2	BFS	–	30 001	2,63
2	Beam	pagoda	9 208	0,89
2	Beam	compuesta	10 790	4,81
3	BFS	–	30 001	4,27
3	Beam	pagoda	14 710	4,06
3	Beam	compuesta	17 918	25,68
4	BFS	–	30 001	1,99
4	Beam	pagoda	7 931	0,84
4	Beam	compuesta	8 842	3,57
5	BFS	–	30 001	6,12
5	Beam	pagoda	18 547	6,45
5	Beam	compuesta	21 305	24,96

Hicimos un experimento adicional sobre la variante 1 con $\beta = 2000$ y 20 reinicios estocásticos. La búsqueda llegó hasta la iteración 31 —justo la profundidad del plan óptimo— y exploró del orden de $9,3 \cdot 10^5$ nodos en 584 s, pero sin dar con la meta. Es un dato bastante elocuente: el algoritmo *sí* llega a la profundidad correcta, pero ninguno de los $\sim 9 \cdot 10^5$ estados que ve es la meta. Con la pagoda como criterio de ordenación, la probabilidad de que la frontera acabe conteniendo el estado meta es prácticamente nula.

1) *Experimento de juego aleatorio*: Para tener otra prueba de lo escaso de soluciones que es este problema, sobre la cruz inglesa lanzamos **80 797 partidas con selección aleatoria uniforme de los movimientos posibles** en cada estado, durante 60 s de cómputo total. La distribución de piezas finales (Tabla IV) tiene la moda en torno a 7 piezas restantes, y ninguna partida llegó a 1 pieza (la mejor se quedó en 2, en 4 ocasiones). Es coherente con lo que ya apuntaba Beasley [5]: las trayectorias que terminan en una sola pieza son extremadamente raras dentro del árbol de juego.

TABLA IV
DISTRIBUCIÓN DEL NÚMERO DE PIEZAS FINALES EN 80 797 PARTIDAS ALEATORIAS SOBRE LA CRUZ INGLESA (60 S DE CÓMPUTO).

Piezas	2	3	4	5	6	7	8
Partidas	4	230	2 520	6 483	13 267	20 117	16 101

Ninguna partida aleatoria llegó a 1 pieza. El mínimo absoluto observado fue 2 (4 de 80 797, $\approx 5 \cdot 10^{-5}$).

Con estos dos resultados —beam search con pagoda fallando incluso a $\beta = 2000$ y cero éxitos sobre $\sim 10^5$ partidas aleatorias— el camino para mejorar el sistema pasaba por cambiar de heurística, no por subir más el haz.

C. Heurística de conectividad: ahora sí

El cuello de botella no era β , era la heurística. Sustituyendo la pagoda por la *heurística de conectividad* —que ordena los estados según el número de componentes conexas— beam search resuelve la cruz inglesa con $\beta = 300$, explorando del orden de 7800 nodos (Tabla V). La meta es la relajada (una única pieza en cualquier casilla), siguiendo la aclaración del profesor.

TABLA V
BEAM SEARCH CON HEURÍSTICA DE CONECTIVIDAD (META RELAJADA, HUECO CENTRAL).

Var.	Cas.	β	Éxito	Movs.	T (s)
1	33	300	sí	31	3,5
2	25	500	no	–	8,6
3	37	800	no	–	59,7
4	25	500	no	–	4,2
5	45	800	no	–	110,7

D. Estudio de la posición del hueco inicial

El profesor valora positivamente que se exploren distintas posiciones del hueco inicial, así que lo hicimos (Tabla VI). El resultado es que **las tres variantes obligatorias (1, 3 y 5) son resolubles** por beam search, aunque V3 y V5 sólo se resuelven partiendo de un hueco situado en un brazo del tablero, no desde el centro. Esto encaja con el problema *complementario* clásico del peg solitaire.

El barrido exhaustivo (probando **todos** los huecos del cuadrante representativo bajo la simetría del tablero) confirma esta influencia (Tabla VI).

TABLA VI
BARRIDO EXHAUSTIVO DEL HUECO INICIAL (BEAM ITERATIVO + CONECTIVIDAD, META RELAJADA). *Min* = MÍNIMO ABSOLUTO DE PIEZAS ALCANZADO ENTRE TODOS LOS HUECOS.

Var.	Tablero	Cas.	Huecos	Resol.	%
1	Octógono	37	13	6	46,2
2	Cruz griega grande	45	16	7	43,8
3	Cruz asimétrica	39	10	10	100,0
4	Cruz griega clásica	33	12	12	100,0
5	Rombo	41	15	3	20,0

En todas las variantes el mínimo absoluto de piezas alcanzado en al menos un hueco fue 1, lo que indica que todas son resolubles desde *algún* hueco. La distinta proporción de huecos resolubles refleja la geometría: V3 y V4 (cruces simétricas) son resolubles desde cualquier posición, mientras que V5 (rombo) sólo lo es desde unas pocas.

De aquí salen un par de lecturas que merece la pena destacar:

- Las dos cruces simétricas (V3, 39 casillas, y V4, la cruz inglesa de 33) son resolubles desde *cualquier* hueco inicial, sea el centro o el extremo de un brazo.

- Las variantes 1 (octógono) y 2 (cruz griega grande) tienen solubilidad parcial: aproximadamente la mitad de los huecos admiten plan.
- El rombo (V5) es el caso más restrictivo: sólo el 20% de los huecos del cuadrante (3/15) admite plan. Tiene sentido: su geometría diagonal limita mucho los saltos posibles desde un mismo punto.
- Ninguna variante es estructuralmente irresoluble: en todas hay al menos algún hueco desde el que beam search encuentra un plan en modo relajado.

La Tabla VII enumera los huecos concretos en los que beam search encuentra plan. Son los candidatos naturales para intentar resolver el problema *complementario*: si partimos del modo estricto desde uno de esos huecos, es razonable esperar terminar con la última pieza precisamente en esa casilla.

TABLA VII
HUECOS INICIALES QUE ADMITEN SOLUCIÓN SEGÚN EL BARRIDO (MODO RELAJADO). LAS COORDENADAS SON (FILA, COLUMNA) 0-INDEXED.

Var.	Huecos resolubles (cuadrante representativo)
1	(0,2), (1,3), (2,0), (2,3), (3,1), (3,2)
2	(0,3), (1,3), (2,3), (3,0), (3,1), (3,2), (3,3)
3	Todos los 10 huecos del cuadrante (100%)
4	Todos los 12 huecos del cuadrante (100%)
5	(1,3), (3,1), (4,2)

El Resultado de *beam search* incluye ahora el atributo `min_piezas_alcanzadas`, que informa de lo cerca que se quedó la búsqueda cuando no llega a la meta. Sirve para distinguir tres situaciones: no hay solución desde ese hueco; sí la hay pero beam search no la encuentra; o el tablero entero es estructuralmente irresoluble. Por ejemplo, que el mínimo absoluto se quede en 4 piezas para *todos* los huecos del diamante es una pista fuerte de irresolubilidad estructural, coherente con lo que da la BFS exhaustiva.

E. Verificación mediante tests unitarios

El sistema está cubierto por una suite de 19 tests unitarios que verifican propiedades formales y empíricas. Entre las cosas que cubren: la consistencia geométrica de los cinco tableros; que la pagoda clásica es realmente una función pagoda formal ($a + b \geq c$ en todas las ternas alineadas); la monotonía no creciente del valor pagoda con cada movimiento legal; que el PDDL que escribe nuestro generador coincide con el que reproduce el lector de `unified-planning`; el modo relajado; que beam search + conectividad resuelve la cruz inglesa en 31 movimientos; y que el parche de `up-fast-downward` se puede aplicar de forma idempotente. Todo se lanza desde `senku/tests/run_tests.py`.

F. Análisis

A partir de los resultados anteriores conviene mirar con detalle cuatro fenómenos.

1) *El espacio de estados del Senku*: BFS no escala más allá de variantes muy pequeñas. La cruz inglesa tiene del orden de 10^8 estados alcanzables, y una BFS exhaustiva, incluso con cota de 30 000 nodos, no llega a profundidad suficiente como para ver una solución. Encaja con el factor de ramificación: cada estado tiene del orden de $\lfloor \text{saltos} \rfloor / 4$ sucesores legales (entre 2 y 12 en los estados intermedios típicos), así que el árbol crece de forma exponencial enseguida.

2) *Admisibilidad vs. informatividad*: La función pagoda es admisible pero poco discriminativa: muchos estados intermedios reciben exactamente el mismo valor, así que el haz no tiene forma de priorizar entre ellos. Esto ejemplifica una distinción que se ve a menudo: una heurística puede ser una cota inferior *ajustada en el límite pero poco útil en la práctica*, simplemente porque todos los estados de la frontera reciben puntuaciones parecidas. Cuando el haz se satura con estados equivalentes, la admisibilidad deja de compensar; vale más cambiar a una heurística que discrimine.

3) *Heurística vs. tamaño del haz*: La elección de heurística pesa más que la anchura del haz: con pagoda no salimos del agujero ni con $\beta = 2000$, mientras que con conectividad resolvemos la cruz inglesa con $\beta = 300$ explorando ≈ 7800 nodos. El cuello de botella es la calidad de la señal, no el tamaño del haz. En números: multiplicar β por 6,7 (de 300 a 2000) no basta para resolver con pagoda, pero cambiar de pagoda a conectividad lo resuelve con un haz casi 7 veces menor. La moraleja es práctica: en un planificador heurístico, invertir esfuerzo en mejorar la heurística suele rendir más que en ampliar el presupuesto de búsqueda.

4) *Influencia del hueco inicial*: La posición del hueco inicial es decisiva. El barrido exhaustivo muestra que la cruz inglesa es resoluble desde todos los huecos considerados (12/12 bajo simetría), pero V2, V3 y V5 sólo admiten plan desde una parte de las posiciones, y el diamante no lo admite desde ninguna. Esto conecta directamente con el problema *complementario* clásico del peg solitaire: la viabilidad de una instancia depende mucho de las invariantes (paridad, coloreado módulo 3, función pagoda) que induce la posición del hueco. Identificar empíricamente qué huecos sí dan plan aporta información estructural de cada variante que no se ve a simple vista mirando el tablero.

5) *Complementariedad con Fast Downward*: Una cosa que merece la pena resaltar es que nuestro sistema *complementa* a Fast Downward más que competir con él. FD resuelve V3 y V4 (cruces simétricas) en pocos segundos, pero no encuentra plan para V1, V2 ni V5 con presupuestos razonables (probamos timeouts de 60–300 s). Beam search con conectividad, en cambio, sí encuentra plan para V1 desde varios huecos iniciales (incluido el del enunciado), para V5 desde tres huecos del cuadrante representativo y, en general, para las configuraciones del barrido en las que existe solución. Combinando los dos enfoques se cubre toda la batería del enunciado: las cruces simétricas las pasamos a Fast Downward (más rápido) y los tableros más densos (octógono, rombo) los resolvemos con beam search + conectividad.

6) *Por qué funciona la conectividad*: La conectividad funciona porque captura una invariante estructural muy fuerte: para llegar a una sola pieza, las piezas restantes tienen que seguir formando una única componente conexas hasta el final. Si una de ellas se queda aislada, el plan ya es imposible. Minimizar el número de componentes conexas en cada paso obliga a beam search a priorizar trayectorias que mantienen el conjunto cohesionado, lo que es una condición casi necesaria (no suficiente) para alcanzar la meta. La pagoda, en cambio, ignora la estructura topológica: dos estados con el mismo número de piezas en posiciones del mismo peso reciben el mismo valor, da igual que uno forme una cadena y el otro varios grupos desconectados.

V. CONCLUSIONES

Hemos montado un sistema completo para resolver el Senku como problema de planificación automática. El dominio PDDL es genérico (vale para cualquier topología, porque la geometría va como hechos del estado inicial) y se integra sin fricción con *unified-planning* y con Fast Downward, igual que en la Práctica 4. Sobre esa base, en Python, están la BFS de la parte común y el beam search de la convocatoria de junio, con varias heurísticas (pagoda, compuesta y, sobre todo, conectividad) y con reinicios estocásticos para diversificar el desempeño.

De los experimentos sacamos tres resultados. El primero: Fast Downward resuelve V3 (cruz asimétrica) en 8 s y V4 (cruz inglesa clásica) en 18 s, pero se atasca en V1, V2 y V5 incluso con timeouts de hasta 300 s. El segundo: beam search guiado por pagoda no resuelve ninguna variante, ni siquiera con $\beta = 2000$; el Senku tiene soluciones demasiado raras (del orden de 10^6 planes en un espacio de 10^8 estados) y la pagoda no las discrimina. El tercero, que es el central: sustituyendo la pagoda por la **heurística de conectividad** —basada en minimizar el número de componentes conexas—, beam search sí resuelve la cruz inglesa (31 movimientos, ≈ 7800 nodos), y eligiendo bien el hueco inicial las tres variantes obligatorias (1, 3 y 5). El estudio del hueco confirma además que V3 y V5 sólo se resuelven partiendo de un hueco en un brazo, no desde el centro.

El barrido sistemático de huecos nos da, además, una clasificación empírica de las variantes: V3 y V4 (cruces simétricas) admiten plan desde cualquier hueco bajo simetría (100%); V1 (octógono) y V2 (cruz griega grande) son parcialmente resolubles (46% y 44%); y V5 (rombo) sólo en un 20% de los huecos. El atributo `min_piezas_alcanzadas` ayuda a separar los casos: en las variantes parcialmente resolubles beam search llega a 1 pieza en al menos un hueco, mientras que en los huecos no resolubles el mínimo se queda en 3–4 piezas.

A. Aplicabilidad metodológica

Más allá del Senku concreto, algunas decisiones del trabajo se pueden reutilizar en otros problemas de planificación con estructura parecida:

- Codificar la geometría como hechos estáticos del `:init` (en lugar de tipos o predicados de adyacencia) viene bien siempre que el tablero pueda variar entre instancias.
- El *beam search iterativo* con β creciente reduce el coste medio cuando unas instancias son fáciles y otras difíciles, sin perder cobertura.
- Heurísticas basadas en propiedades *estructurales* del estado (componentes conexas, propiedades topológicas) suelen ganar a las *cuantitativas* (sumas ponderadas tipo pagoda) cuando el espacio de soluciones es muy raro.

B. Líneas futuras

Si tuviéramos que seguir trabajando en esto, los siguientes pasos que nos parecen más interesantes son:

- Añadir *look-ahead* de uno o dos niveles para evaluar la heurística sobre los sucesores inmediatos en lugar de sobre el estado actual, lo que mitigaría el efecto *horizon* típico de las heurísticas locales.
- Combinar beam search con búsquedas locales estocásticas (*hill climbing* con reinicios aleatorios), que pueden ayudar en las regiones donde el haz se queda atascado.
- Integrar de forma más estrecha con Fast Downward: por ejemplo, comparar la calidad de los planes encontrados por beam search frente a los que da FD cuando llega (en Senku, al ser coste uniforme, los planes con menos saltos son los óptimos).
- Probar heurísticas específicas para el Senku basadas en *packings* y patrones combinatorios documentados en la literatura clásica del juego [5].
- Aplicar el mismo enfoque a otros solitarios sobre rejilla (Mahjong, juegos de eliminación), donde la conectividad también tiene un papel importante.
- Aprovechar la simetría del tablero para reducir el espacio de búsqueda. La cruz inglesa tiene grupo D_4 (rotaciones y reflexiones), lo que permitiría factorizar por clases de equivalencia y bajar el factor de ramificación efectivo.

C. Reproducibilidad

Todo el código, los ficheros PDDL, los notebooks de experimentación y los CSV con los resultados están en el directorio `senku/` del entregable. Para reproducir los experimentos basta con ejecutar los scripts `senku/scripts/experimento_*.py`. La suite de 19 tests unitarios (todos en verde) cubre tanto las propiedades formales (validez de la pagoda, monotonía, etc.) como la solubilidad efectiva de la cruz inglesa con beam search + conectividad.

D. Uso de inteligencia artificial generativa

Como pide el enunciado, declaramos el uso de un asistente generativo (Claude, modelo Opus 4.7) durante el desarrollo del trabajo. Lo hemos usado, en concreto, para:

- 1) Generar una estructura inicial del repositorio (módulos Python, organización de directorios), que después hemos revisado y adaptado.

- 2) Discutir formulaciones alternativas del dominio PDDL y sus ventajas/inconvenientes; eso influyó en la decisión final de modelar la geometría como hechos `salto` del estado inicial.
- 3) Implementar las heurísticas, los algoritmos de búsqueda y los scripts de experimentación a partir de descripciones funcionales nuestras. El código resultante lo hemos revisado, entendido y, en varios casos, modificado.
- 4) Ayudarnos con la redacción de la memoria y la presentación, pero manteniendo nosotros la decisión editorial final.
- 5) Detectar y arreglar el bug de codificación de `up-fast-downward 0.5.2` con el módulo `parche_fd.py`.

Las peticiones que le hemos hecho han sido del tipo: “implementa el beam search siguiendo el pseudocódigo del enunciado”, “diseña una heurística que capture la conectividad de las piezas restantes”, “escribe un script que recorra todos los huecos iniciales bajo simetría”, o “corrige el error de decodificación que aparece al llamar a Fast Downward en Windows”. En la defensa, los dos miembros del grupo somos capaces de explicar y justificar cualquier decisión técnica y editorial del entregable.

REFERENCIAS

- [1] G. Chaves Benítez, “Planificación automática aplicada al Senku. Propuesta de trabajo,” Inteligencia Artificial — Ingeniería del Software, Universidad de Sevilla, curso 2025/2026.
- [2] Planning Wiki, “Planning Domain Definition Language,” <https://planning.wiki/>, consultado en mayo de 2026.
- [3] A. Micheli *et al.*, “Unified-Planning: A Library Making Planning Technology Accessible,” en *Proc. of ICAPS*, 2022. Documentación: <https://unified-planning.readthedocs.io>.
- [4] M. Helmert, “The Fast Downward Planning System,” *Journal of Artificial Intelligence Research*, vol. 26, pp. 191–246, 2006.
- [5] J. C. Beasley, *The Ins and Outs of Peg Solitaire*, Oxford University Press, 1985.
- [6] S. Russell y P. Norvig, *Artificial Intelligence: A Modern Approach*, 4ª ed., Pearson, 2020. Capítulos 3 y 4 sobre búsqueda en espacio de estados y búsqueda informada.
- [7] M. Ghallab *et al.*, “PDDL—The Planning Domain Definition Language,” Yale Center for Computational Vision and Control, Technical Report CVC TR-98-003, 1998.
- [8] M. Ghallab, D. Nau y P. Traverso, *Automated Planning: Theory and Practice*, Morgan Kaufmann, 2004.
- [9] D. S. Hochba (ed.), *Approximation algorithms for NP-hard problems*, PWS Publishing, 1997. Sobre la complejidad computacional de problemas combinatorios.
- [10] E. Berlekamp, J. H. Conway y R. Guy, *Winning Ways for your Mathematical Plays*, vol. 4, A. K. Peters, 2004. Capítulo sobre peg solitaire con la introducción original de la función pagoda por J. H. Conway.
- [11] N. G. de Bruijn, “A solitaire game and its relation to a finite field,” *Journal of Recreational Mathematics*, vol. 5, pp. 133–137, 1972. Análisis algebraico clásico del peg solitaire mediante argumentos de paridad y coloreado.